Faculty of Engineering and Technology
Object Oriented Programming (CEUE203)
A.Y 2026 – 27 Semester – 3
Practical List

Practical Number		CO/PO
1	**Part A — Practice Programs (Hour 1)** Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home. **1. Vending machine (VendingMachine.java, main in this file)** (a) Define an enum Coin with constants ONE, TWO, FIVE, TEN. (b) In main, set a snack price of 15 and a running total of 0; create a Scanner. (c) Loop: read a coin name, use a switch expression to convert the Coin to its value (ONE→1, TWO→2, FIVE→5, TEN→10), add it to the total, and print the total so far. (d) Stop the loop once the total reaches 15 or more. (e) Print the change to return (total − 15). **Expected output:** *TEN then FIVE prints "Paid. Change: 0"; TEN then TEN prints "Paid. Change: 5".* **2. Toll booth (TollBooth.java, main in this file)** (a) Define a record Vehicle(String number, String type). (b) In main, keep a running toll total and three counters (bike, car, truck). (c) Loop until the user types "done" for the number: build a Vehicle, use a switch expression on its type for the toll (bike→20, car→50, truck→150), add to the total, and increment that type's counter. (d) After the loop, print the total toll and the type with the highest count. **Expected output:** *two cars + one bike → "Total toll: 120", "Most frequent: car".* **3. (For Advanced Learners – home) Rock-Paper-Scissors-Lizard-Spock (RPSLS.java)** (a) Define an enum Move with ROCK, PAPER, SCISSORS, LIZARD, SPOCK. (b) Write winner(Move a, Move b) returning 1 if a beats b, -1 if b beats a, 0 for a tie — use a switch expression listing what each move beats (Scissors cut Paper, Paper covers Rock, Rock crushes Lizard, Lizard poisons Spock, Spock smashes Scissors, and the five mirror cases). (c) In main, play 5 rounds: pick a random computer Move, read the player's Move, call winner(), print the round result and keep score. (d) After 5 rounds, print the overall winner. **Expected output:** *each round prints both moves and the round winner; ends with a "You win 3–2" summary.* **Part B — Project Work (Hour 2): MiniBank** MiniBank is the console banking application you will build across the whole semester. In this first lab you set up your tools and create the project and its menu shell — the screen the user navigates. Every later lab attaches one real banking feature behind these menu options. 1. Set up your environment: open the project in GitHub Codespaces (or install JDK 17+ and an IDE locally), and create a private GitHub repository named oop-minibank-<rollno>; add the faculty as a collaborator. 2. Create a public class named MiniBank that contains the main method. 3. Define a record named BankInfo with two String fields, name and branch (a record is a short, read-only class; Java auto-generates its constructor and field accessors). Print one BankInfo object as the application header. 4. Define an enum named MenuOption with the constants OPEN_ACCOUNT, DEPOSIT, WITHDRAW, TRANSFER and EXIT — these are the actions MiniBank will offer. 5. Display the numbered menu to the user and read the chosen number with a Scanner. 6. Use a switch expression on the choice to print a short placeholder for that option (for example, "Deposit — to be implemented in a later lab"). Keep showing the menu in a loop until the user selects EXIT, then print a goodbye message. **Key Questions / Analysis / Interpretation to be evaluated during/after Implementation** 1. What is the role of the JVM, and what file does the javac compiler produce?	**CO1** PO1, PO2, PO3

Practical Number		CO/PO
	2. How does a switch expression differ from a traditional switch statement? 3. Why is an enum a good choice for a fixed set of menu options, and a record for read-only data? **Supplementary Problems –** • Re-prompt the user when an invalid menu number is entered. • Add a menu option that prints the bank's working hours. **Key Skills to be addressed –** Java program structure; compiling and running with javac/java; enum and record; switch expression; reading input with Scanner; basic git (commit, tag). **Applications –** The starting point of every menu-driven console application — ATM front-ends, point-of-sale terminals, kiosk software. **Learning Outcome –** The student can set up the Java toolchain and write, compile, run and version-control a basic interactive program. (CO1)	
	Dataset / Test Data (Source and Description, if applicable): No external dataset. The user types menu choices (e.g., 1, 2, 3, 4) at runtime.	
	Tools / Technology To Be Used: JDK 17+, GitHub Codespaces (or a local IDE), command line, git.	
	• **Total Hours of Problem Definition Implementation:** 2 hours — Hour 1: Part A practice programs; Hour 2: Part B project work. • **Total Hours of Engagement** = time to implement the solution + modification time (if necessary) + testing by the faculty member: approximately 2.5 hours.	
	Post Laboratory Work Description: Commit and tag lab-01. Write a short README describing how to compile and run. Verify the menu loops correctly for every option, including EXIT.	
	Rubrics (Practical Evaluation / Viva): Part A practice programs – 20%. Part B project (80%): Program compiles and menu runs – 40%; correct use of enum, record and switch expression – 30%; code clarity and naming conventions – 15%; viva (JVM, switch expression, record) – 15%.	
	Expected Input / Process / Output **Input:** The user enters a menu number, for example 2 (Deposit). **Process:** main reads the number with Scanner; the switch expression maps it to the matching MenuOption and prints the placeholder; the loop then redraws the menu. **Expected Output:** The header and menu are shown; entering 2 prints "Deposit — to be implemented in a later lab"; entering the EXIT number prints a goodbye message and stops the program.	